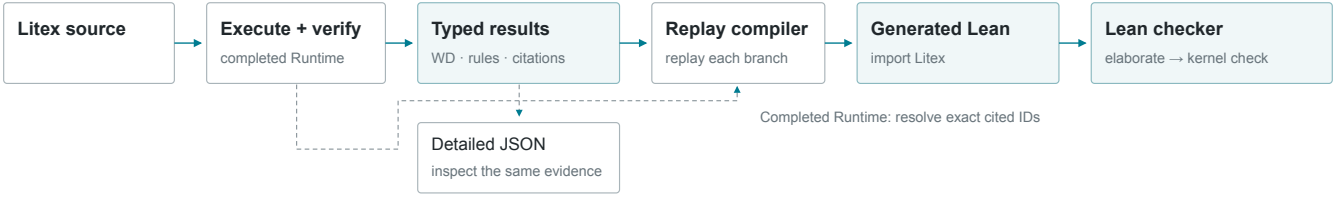
From checked evidence to a Lean proof

Preserve the source objects, construction conditions and selected proof route.



JSON is a view of the results. The current compiler consumes typed results + Runtime.

a Inspect the evidence

Actual source and Detailed fields

```
forall a C:
    a + 0 = a
```

Introduce a complex member

a ∈ ℂ, stored as f15

Check the addition

a ∈ ℂ and 0 ∈ ℂ certify a + 0

Selected equality proof

```
"searched_proof": {
  "type": "builtin_rule",
  "rule": "Calculation",
  "mode": "rational" }
```

local f1 → quantified f2

b Replay the selected route

Recorded evidence → emitted terms

Membership citation f15

```
h_15 : Litex.In … Litex.C
```

Addition WD evidence

```
Litex.add … h_15
  (Litex.numberInC …)
```

Calculation / rational

```
Litex.NativeBridge.
  sameOfDenoteNumber …
  (by ring!)
```

Stored f1 / exported f2

```
have h_1 … / theorem h_2 …
```

Fixed normalization inside the selected adapter.
The final goal does not select a replacement proof.

c What Litex.lean supplies

Lean meaning and proved bridge laws

1 Objects keep their conditions

Obj α = value + WD evidence

`Representation M α · denote`

2 Facts have explicit meanings

In a A: membership · Same a b: equal denotations

`Semantics · In · Same`

3 Bridge laws are proved in Lean

native equality + denotation witnesses → Same

`NativeBridge.sameOfDenoteNumber`

M : Semantics stays explicit.
The theorem assumes its interpretation laws.

import Litex provides these declarations and proofs.

Proved library + generated terms → independently checked Lean theorem

Optional next step: handwritten Adapter → ordinary Lean / Mathlib interface.